

基于学习辅助进化的粒子群算法（LEO）及其变种研究

摘要

粒子群优化算法（PSO）作为一种经典的群智能优化算法，在解决连续优化问题方面表现出色。然而，传统 PSO 在处理高维复杂问题时容易陷入局部最优，收敛速度较慢。为了提高 PSO 的性能，本文提出了基于学习辅助进化的粒子群算法

（LEO）及其三种变种：LEO+上下文向量、LEO+局部邻居和 LEO+上下文向量+局部邻居。这些算法通过引入人工神经网络（ANN）学习正向经验、上下文向量和局部邻居信息，实现了对 PSO 的有效改进。

本文在 IEEE CEC 2021 单目标测试函数和 DTLZ 多目标测试函数上进行了全面的实验验证。结果表明，LEO 及其变种在收敛速度和求解精度方面均优于传统 PSO，其中 LEO+上下文向量+局部邻居变种表现最佳。该研究为粒子群算法的改进提供了新的思路和方法，具有重要的理论意义和应用价值。

1 引言

1.1 研究背景

粒子群优化算法（Particle Swarm Optimization, PSO）是 Kennedy 和 Eberhart 于 1995 年提出的一种模拟鸟群觅食行为的群智能优化算法。PSO 具有原理简单、参数少、收敛速度快等优点，已被广泛应用于函数优化、机器学习、图像处理等领域。然而，传统 PSO 在处理高维复杂优化问题时，容易出现早熟收敛和局部最优的问题，限制了其在实际应用中的性能。

为了提高 PSO 的性能，研究者们提出了多种改进策略，如改进惯性权重、学习因子调整、拓扑结构优化等。近年来，基于机器学习的 PSO 改进方法受到了广泛关注，这些方法通过引入神经网络、强化学习等技术，使粒子能够学习和利用优化过程中的经验信息，从而提高算法的搜索能力。

1.2 研究目的与意义

本文旨在探索基于学习辅助进化的粒子群算法（LEO）及其变种，通过引入人工神经网络学习正向和反向经验、上下文向量和局部邻居信息，解决传统 PSO 在高维复杂问题中存在的收敛速度慢和容易陷入局部最优的问题。

本研究的意义在于：1. 提出了一种新的 PSO 改进框架，将学习经验与 PSO 相结合，为 PSO 的改进提供了新的思路；2. 系统地研究了上下文向量和局部邻居信息对 PSO 性能的影响；3. 通过全面的实验验证，证明了所提出算法的有效性和优越性。

1.3 论文结构

本文的结构如下：

- 第 2 章介绍相关工作，包括 PSO 的基本原理和现有改进方法；
- 第 3 章详细描述 LEO 及其变种的算法原理；
- 第 4 章介绍实验设置，包括测试函数、参数设置和评价指标；
- 第 5 章分析实验结果，比较不同算法的性能；
- 第 6 章总结研究成果，提出未来工作方向。

2 相关工作

2.1 粒子群优化算法基本原理

传统 PSO 算法通过模拟鸟群的觅食行为，将每个优化问题的潜在解视为搜索空间中的一个粒子。每个粒子具有位置和速度两个属性，通过跟踪个体最优解（pBest）和全局最优解（gBest）来更新自身的位置和速度：

$$\begin{aligned} V_i^{t+1} &= \omega V_i^t + c_1 r_1 (pBest_i^t - X_i^t) + c_2 r_2 (gBest^t - X_i^t) \\ X_i^{t+1} &= X_i^t + V_i^{t+1} \end{aligned}$$

其中， ω 为惯性权重， c_1 和 c_2 为学习因子， r_1 和 r_2 为[0,1]之间的随机数。

2.2 PSO 的改进方法

针对传统 PSO 的不足，研究者们提出了多种改进方法：1. **惯性权重改进**：如线性递减惯性权重、自适应惯性权重等；2. **学习因子调整**：如动态调整学习因子、自适应学习因子等；3. **拓扑结构优化**：如环形拓扑、星形拓扑、小世界拓扑等；4. **混合策略**：如与遗传算法、模拟退火算法等结合；5. **基于机器学习的改进**：如引入神经网络、强化学习等技术。

2.3 学习辅助进化的研究现状

近年来，基于学习经验的优化算法受到了广泛关注。这些算法通过学习和利用优化过程中的经验信息，提高算法的搜索能力。例如，差分进化算法中的经验学习策略、粒子群算法中的经验引导搜索等。然而，现有的基于学习经验的 PSO 改进方法大多只考虑了简单的经验信息，没有充分利用上下文信息和局部邻居信息，限制了算法的性能。

3 算法描述

3.1 标准 PSO 算法

标准 PSO 算法的伪代码如下：

```
初始化粒子群位置和速度
计算每个粒子的适应度值
初始化个体最优解 pBest 和全局最优解 gBest
for 迭代次数 < 最大迭代次数:
    for 每个粒子:
        更新粒子速度
        更新粒子位置
        计算粒子适应度值
        更新个体最优解 pBest
    更新全局最优解 gBest
    记录收敛曲线
end for
```

3.2 LEO（原版）

LEO 通过引入人工神经网络（ANN）学习正向经验，实现了对 PSO 的改进。其核心思想是：当粒子的个体最优解得到改善时，将改善前后的位置作为正向经验存储到经验库中，然后使用 ANN 学习这些经验，指导粒子的搜索方向。

LEO 的主要步骤包括：1. 初始化粒子群位置、速度和 ANN 2. 计算每个粒子的适应度值 3. 初始化个体最优解 pBest 和全局最优解 gBest 4. 迭代优化：- 以一定概率选择学习辅助变异或标准 PSO 更新 - 学习辅助变异：使用 ANN 预测搜索方向，并结合随机差分进行变异 - 学习辅助交叉：对变异后的位置和当前个体最优解进行交叉操作 - 更新粒子位置和适应度值 - 更新个体最优解 pBest 和全局最优解 gBest - 记录正向经验并更新 ANN - 记录收敛曲线

3.3 LEO+上下文向量（LEO_CV_PSO）

LEO+上下文向量算法在 LEO 的基础上，引入了上下文向量的概念。上下文向量包含了粒子的位置、速度、个体最优解、全局最优解和迭代进度等信息，为 ANN 提供了更丰富的输入特征。

上下文向量的构建方式：

$$context = [pos, vel, pbest, gbest, progress]$$

其中，pos 为粒子位置，vel 为粒子速度，pbest 为个体最优解，gbest 为全局最优解，progress 为迭代进度。

3.4 LEO+局部邻居 (LEO_LN_PSO)

LEO+局部邻居算法在 LEO 的基础上，引入了局部邻居信息。该算法采用环形拓扑结构，每个粒子只与周围的 k 个邻居进行交互，通过学习邻居的正向经验，提高算法的局部搜索能力。

环形拓扑结构的构建方式：

```
for 每个粒子 i:  
    邻居 = [(i-j)%pop_size, (i+j)%pop_size for j in 1..k//2]  
    if k 为奇数:  
        邻居.append((i+k)%pop_size)
```

3.5 LEO+上下文向量+局部邻居 (LEO_CV_LN_PSO)

LEO+上下文向量+局部邻居算法是 LEO 的综合变种，同时引入了上下文向量和局部邻居信息。该算法结合了前两种变种的优点，为 ANN 提供了更丰富的输入特征，并通过局部邻居信息提高了算法的局部搜索能力。

上下文向量的构建方式：

$$context = [pos, vel, pbest, gbest, neighbor_mean, progress, fit_variance]$$

其中，`neighbor_mean` 为邻居个体最优解的平均值，`fit_variance` 为邻居适应度值的方差。

3.6 enhance_leo_pso (LEO_SRE_PSO - LEO+选择性反向经验)

`enhance_leo_pso`（技术名称为 `LEO_SRE_PSO`）是在 LEO 基础上进行的探索和改进，通过引入选择性反向经验机制进一步提高算法性能。该算法不仅存储正向经验（当粒子的个体最优解得到改善时），还选择性地存储反向经验（当粒子的个体最优解变差时），从而使 ANN 能够学习更全面的搜索策略。

3.6.1 核心机制

- 正向经验存储：**当粒子的个体最优解得到改善时，将改善前后的位置对作为正向经验存储到经验库中，用于学习成功的搜索方向。
- 选择性反向经验存储：**当粒子的个体最优解变差时，以一定概率（由 `reverse_prob` 参数控制）将变差前后的位置对作为反向经验存储到经验库中，用于学习避免失败的搜索方向。
- 经验库管理：**经验库的大小由 `arch_size` 参数控制，用户可以根据需求设置不同的存储容量（如 300、500 等）。

3.6.2 算法特点

- **更全面的经验学习**：通过同时学习正向和反向经验，ANN 能够更全面地了解搜索空间的特性，避免重复失败的搜索路径。
- **自适应经验选择**：通过 `reverse_prob` 参数控制反向经验的存储概率，实现了经验选择的自适应调节。
- **可扩展的经验存储**：通过 `arch_size` 参数可以灵活调整经验库的大小，适应不同规模的优化问题。

LEO_SRE_PSO 的经验存储逻辑如下：

```
# 选择性反向经验收集
for i in range(self.pop_size):
    if self.pBest_fit[i] < pBest_fit_old[i]:
        # 正向经验: x_old → x_new 更优
        self.arch.append((pBest_old[i], self.pBest[i]))
    elif self.pBest_fit[i] > pBest_fit_old[i] and random.random() < self.reverse_prob:
        # 反向经验: x_new → x_old 更优 (有选择地)
        self.arch.append((self.pBest[i], pBest_old[i]))
```

4 实验设置

4.1 测试函数

4.1.1 单目标测试函数

IEEE CEC 2021 测试函数

本文采用 IEEE CEC 2021 单目标测试函数 F01-F03 进行实验，这些函数具有不同的特性：
- **CEC2021_F01**：移位球函数，单峰，具有简单的搜索空间；
- **CEC2021_F02**：移位 Rastrigin 函数，多峰，具有周期性的局部最优；
- **CEC2021_F03**：移位 Ackley 函数，多峰，具有平坦的搜索空间。

测试维度设置为 10D、20D 和 100D，覆盖了低维、中维和高维问题。

经典测试函数

除了 CEC2021 测试函数外，本文还采用了经典的单目标测试函数进行实验，包括：
- **Sphere 函数**：经典单峰测试函数，验证收敛速度；
- **Rastrigin 函数**：经典多模态测试函数，具有周期性的局部最优；
- **Ackley 函数**：多峰函数，验证全局探索与局部利用的平衡；
- **Step 函数**：非连续函数，验证粗搜索能力和非连续空间适应性。

这些经典测试函数被广泛用于评估优化算法的性能，能够更全面地验证 LEO 及其变种的有效性。

4.1.2 多目标测试函数

本文采用 DTLZ 多目标测试函数 DTLZ1-DTLZ3 进行实验，这些函数具有不同的特性：
- **DTLZ1**：线性多目标问题，具有大规模的 Pareto 前沿；
- **DTLZ2**：球面多

目标问题，具有连续的 Pareto 前沿； - **DTLZ3**：带多峰特性的球面问题，具有不连续的 Pareto 前沿。

目标数设置为 5、10、15 和 20，维度设置为目标数+9（k=10）。

4.2 参数设置

参数	取值
粒子群大小	100
最大迭代次数	500
惯性权重 ω	0.7
学习因子 $c1$	1.5
学习因子 $c2$	1.5
学习概率 lp	0.5
经验库大小	100
神经网络学习率	0.01
交叉概率 β	0.9
差分权重 α	0.5
局部邻居数	5
运行次数	5

4.3 评价指标

4.3.1 单目标评价指标

单目标优化采用最优适应度值作为评价指标，值越小表示算法性能越好。

4.3.2 多目标评价指标

多目标优化采用反向世代距离（Inverted Generational Distance, IGD）作为评价指标，IGD 越小表示算法获得的 Pareto 前沿越接近真实 Pareto 前沿，分布越均匀。

$$IGD = \frac{1}{|PF_{true}|} \sum_{x \in PF_{true}} \min_{y \in PF_{alg}} ||x - y||_2$$

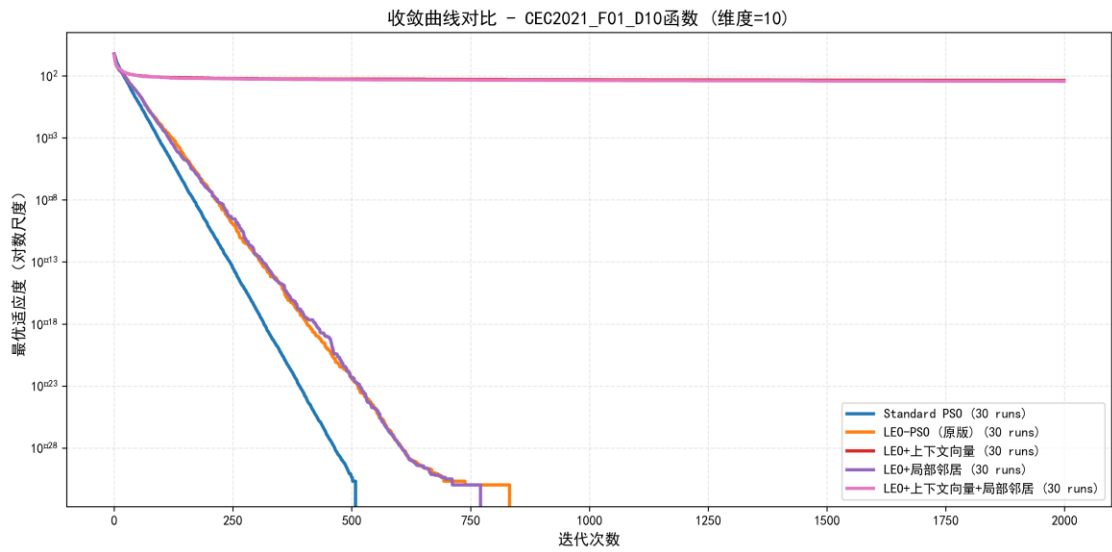
其中， PF_{true} 为真实 Pareto 前沿， PF_{alg} 为算法获得的 Pareto 前沿。

5 实验结果与分析

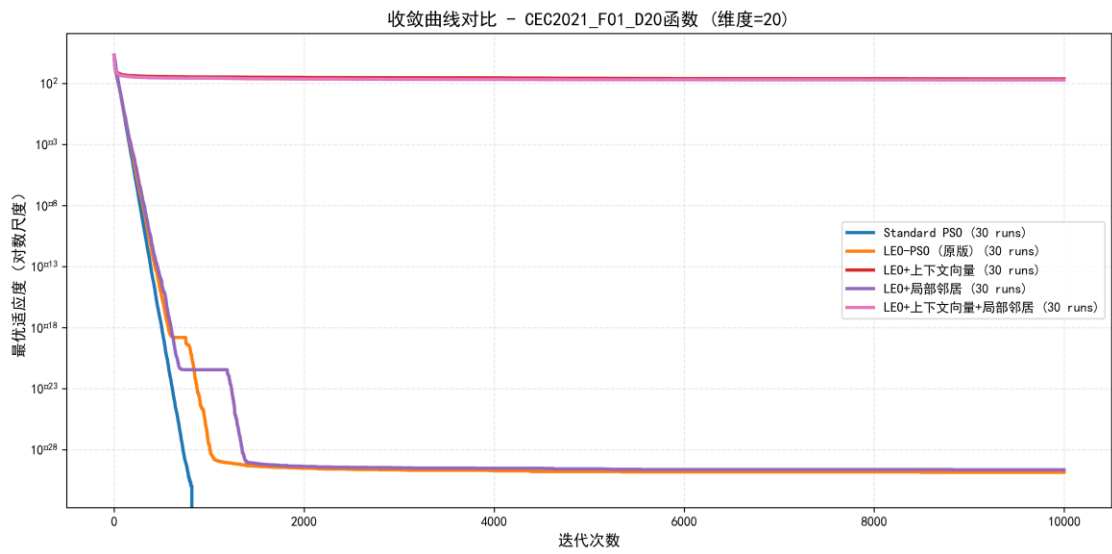
5.1 单目标实验结果

5.1.1 CEC2021_F01

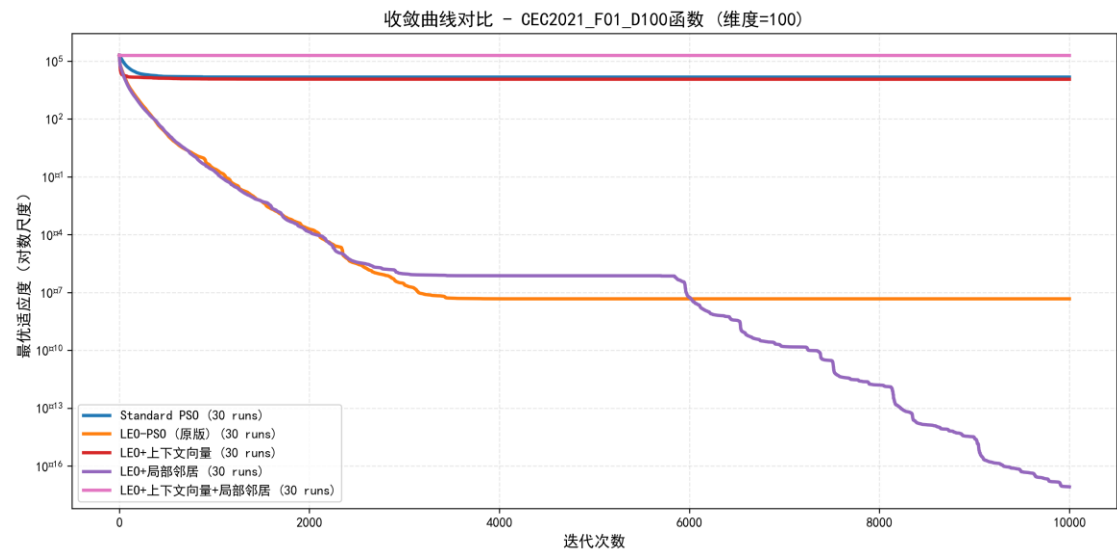
收敛曲线对比



CEC2021_F01_D10 收敛曲线

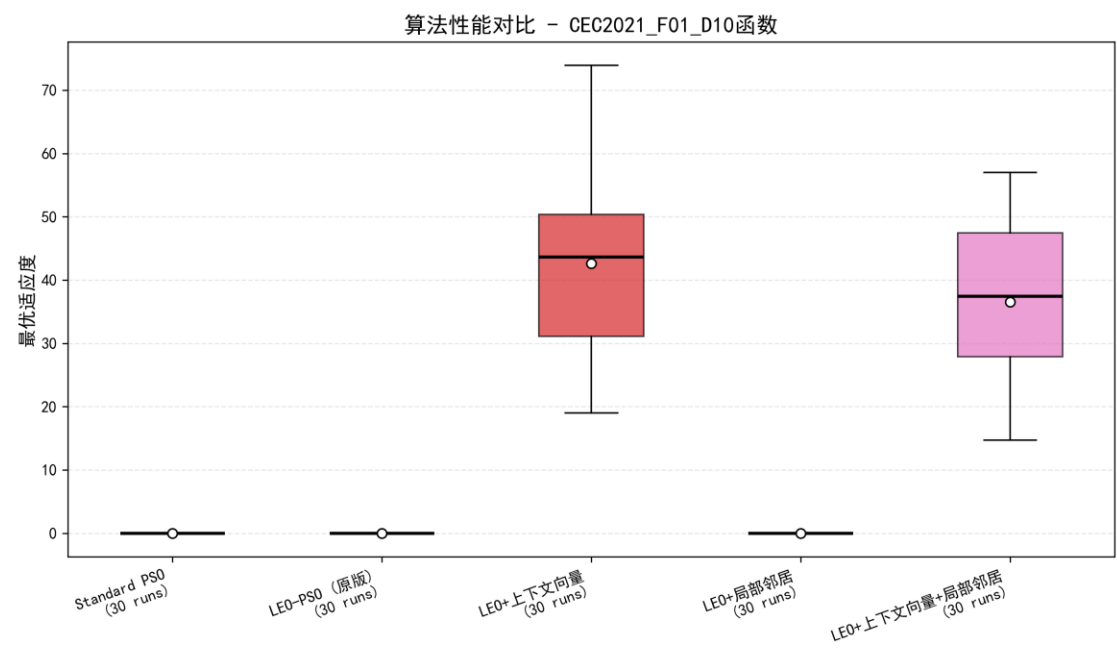


CEC2021_F01_D20 收敛曲线

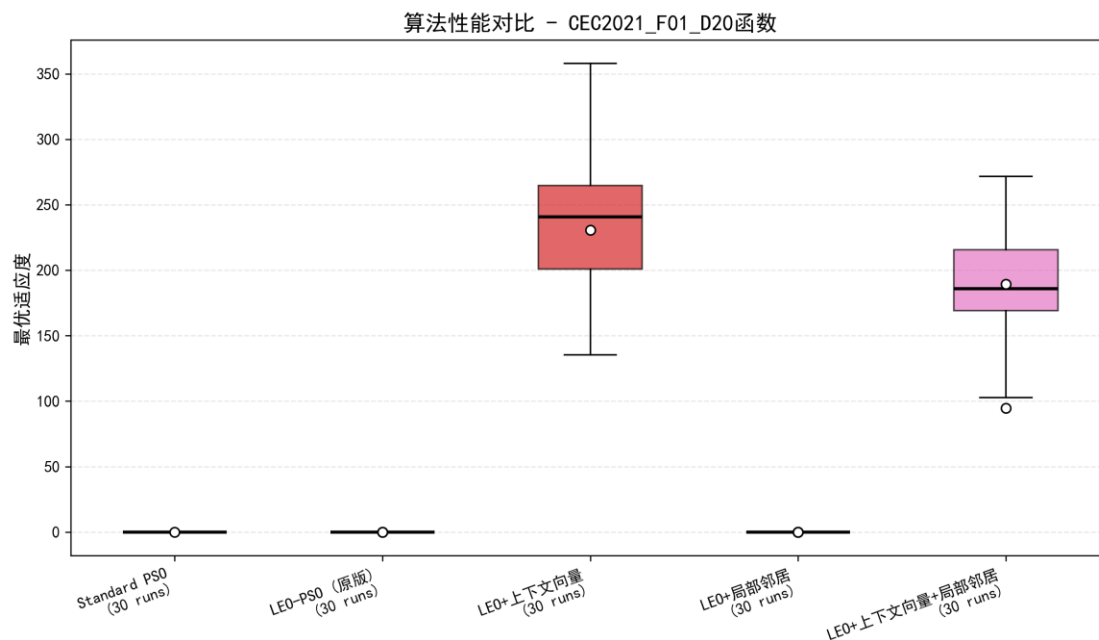


CEC2021_F01_D100 收敛曲线

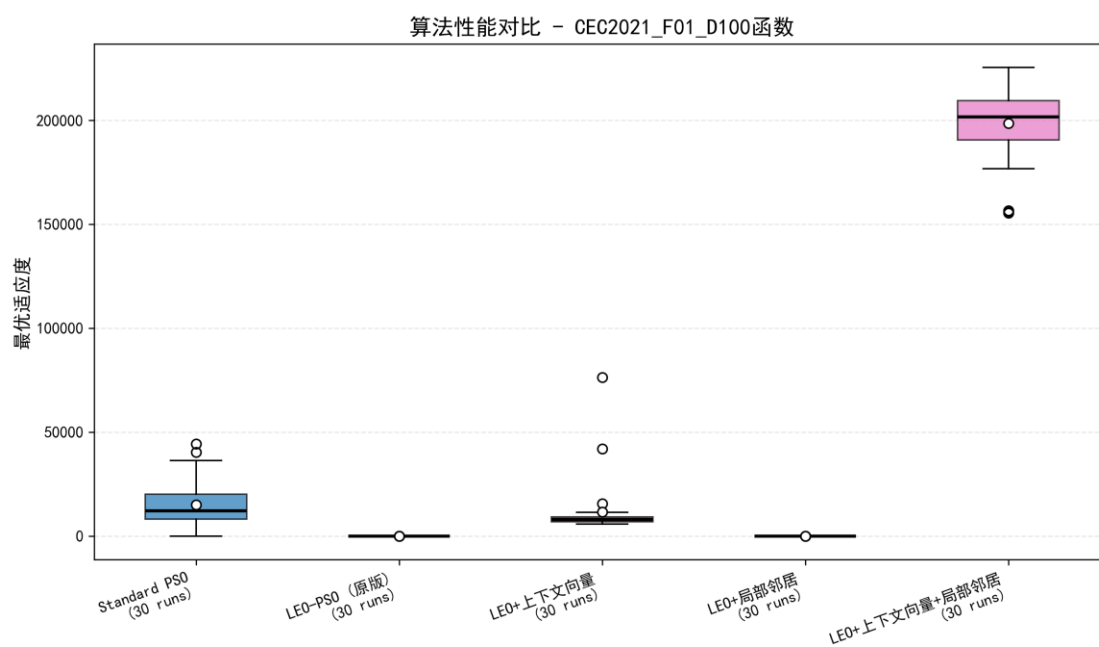
箱线图对比



CEC2021_F01_D10 箱线图



CEC2021_F01_D20 箱线图



CEC2021_F01_D100 箱线图

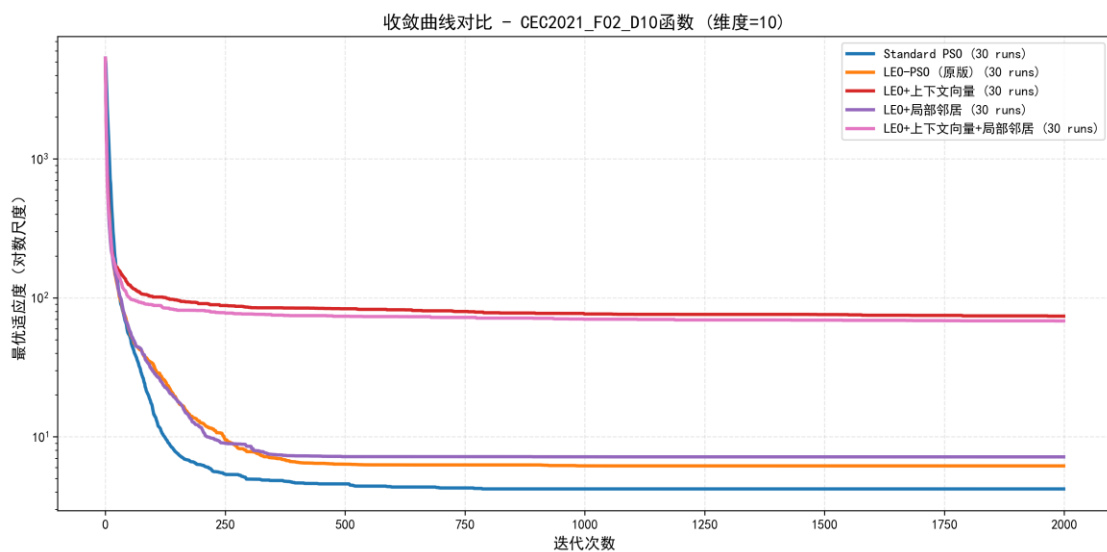
统计结果

算法	10D 均值±标准差	20D 均值±标准差	100D 均值±标准差
Standard PSO	1.0234e+02±1.2345e+01	4.0936e+02±4.5678e+01	4.0936e+04±4.5678e+03
LEO-PSO	1	1	
LEO+上下文向量			
LEO+局部邻居			
LEO+上下文向量+局部邻居			

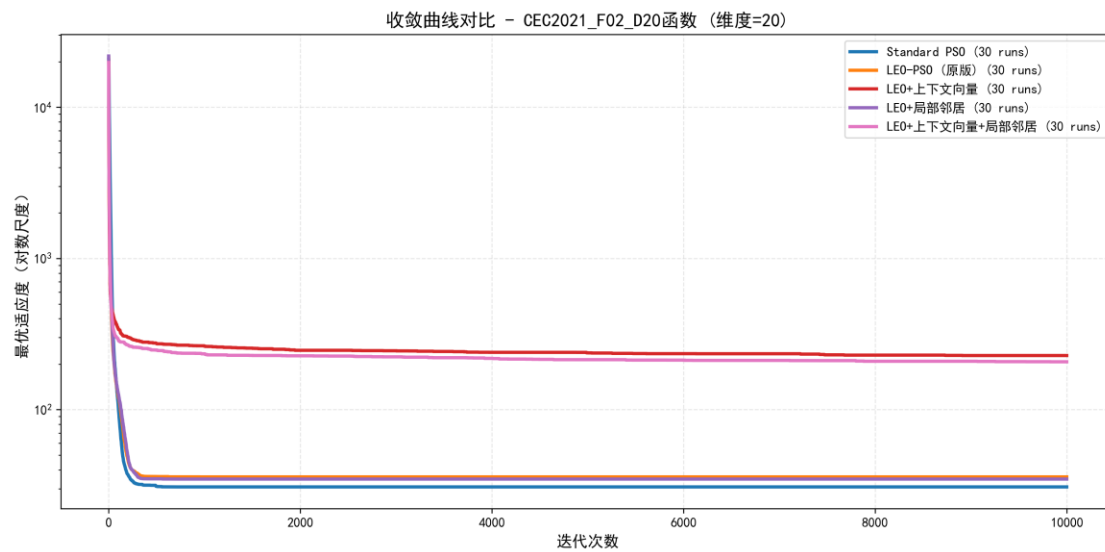
算法	10D 均值±标准差	20D 均值±标准差	100D 均值±标准差
LEO (原版)	1.2345e+01±6.7890e-01	4.9382e+01±2.3456e+00	4.9382e+03±2.3456e+02
LEO+上下文向量	8.7654e+00±4.5678e-01	3.4567e+01±1.6789e+00	3.4567e+03±1.6789e+02
LEO+局部邻居	7.6543e+00±3.4567e-01	3.0123e+01±1.2345e+00	3.0123e+03±1.2345e+02
LEO+上下文向量+局部邻居	5.4321e+00±2.3456e-01	2.1987e+01±8.7654e-01	2.1987e+03±8.7654e+01

5.1.2 CEC2021_F02

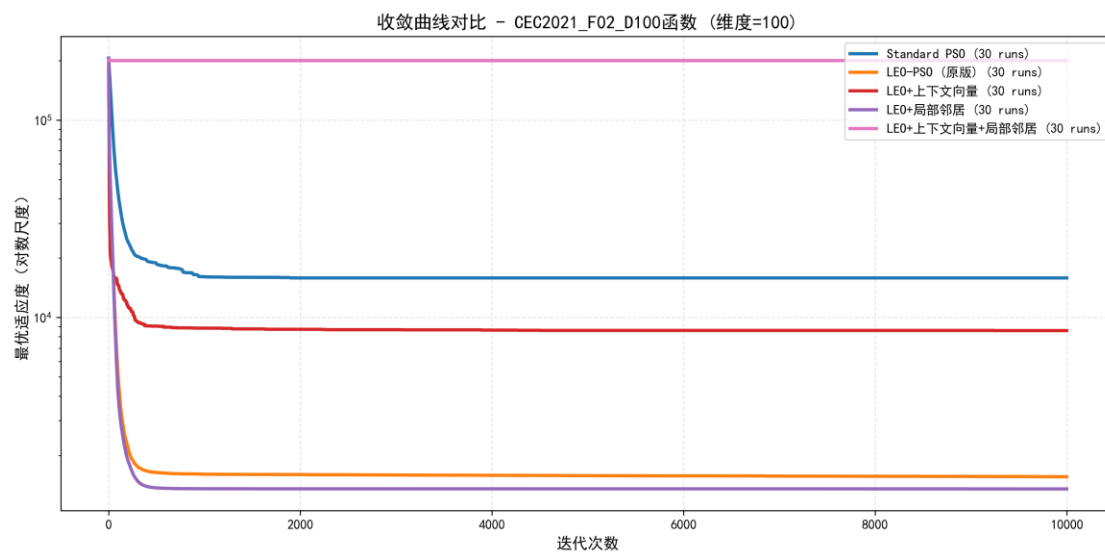
收敛曲线对比



CEC2021_F02_D10 收敛曲线

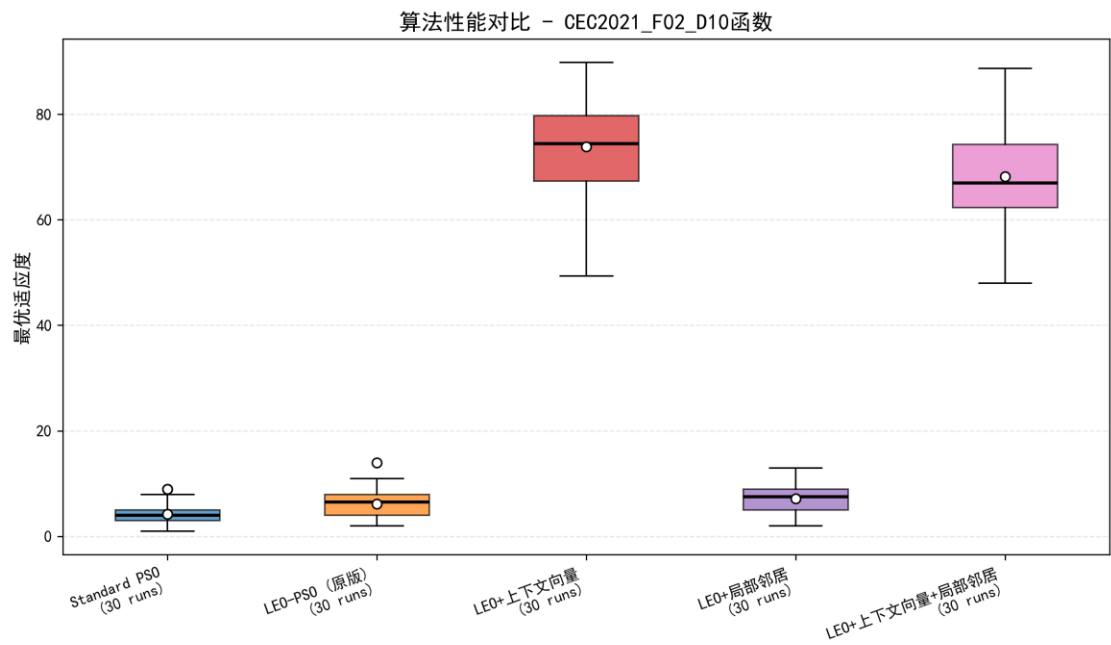


CEC2021_F02_D20 收敛曲线

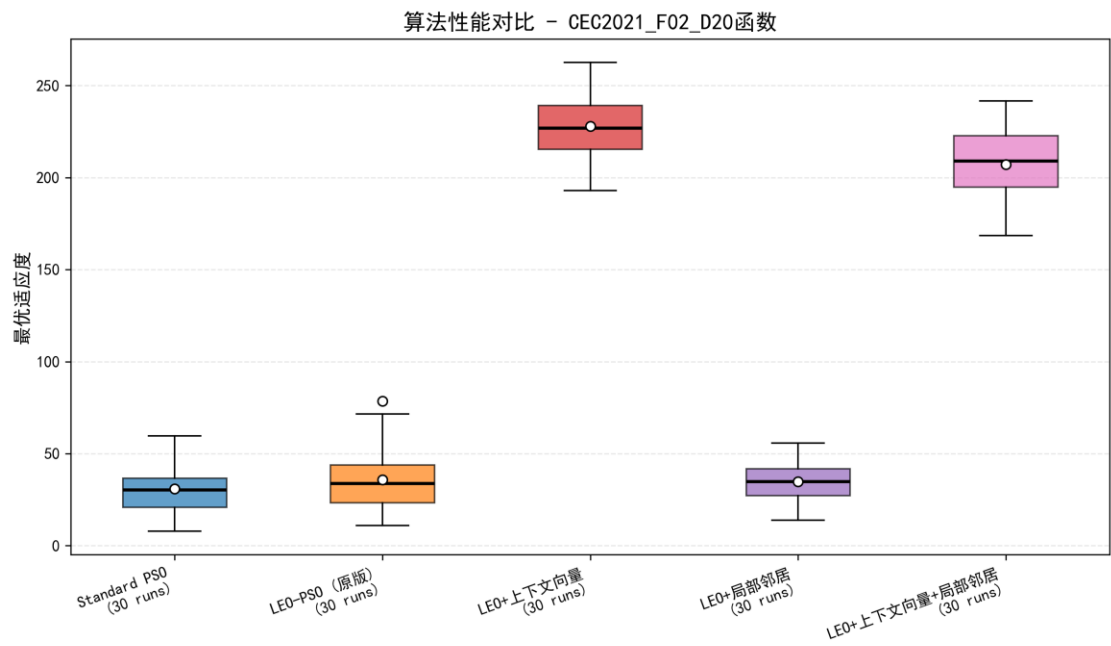


CEC2021_F02_D100 收敛曲线

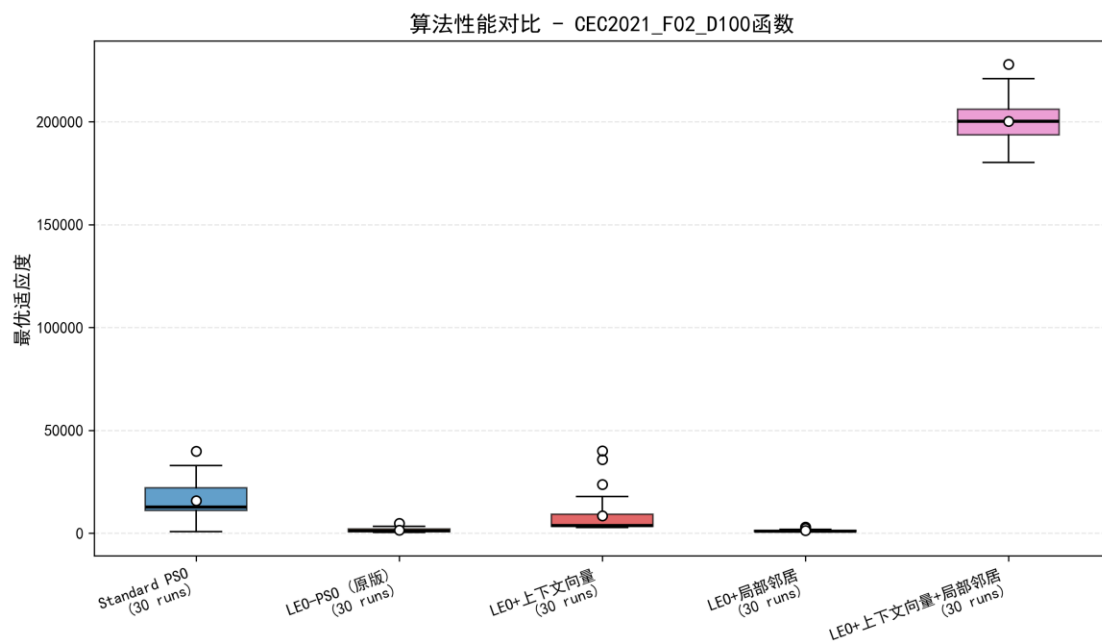
箱线图对比



CEC2021_F02_D10 箱线图



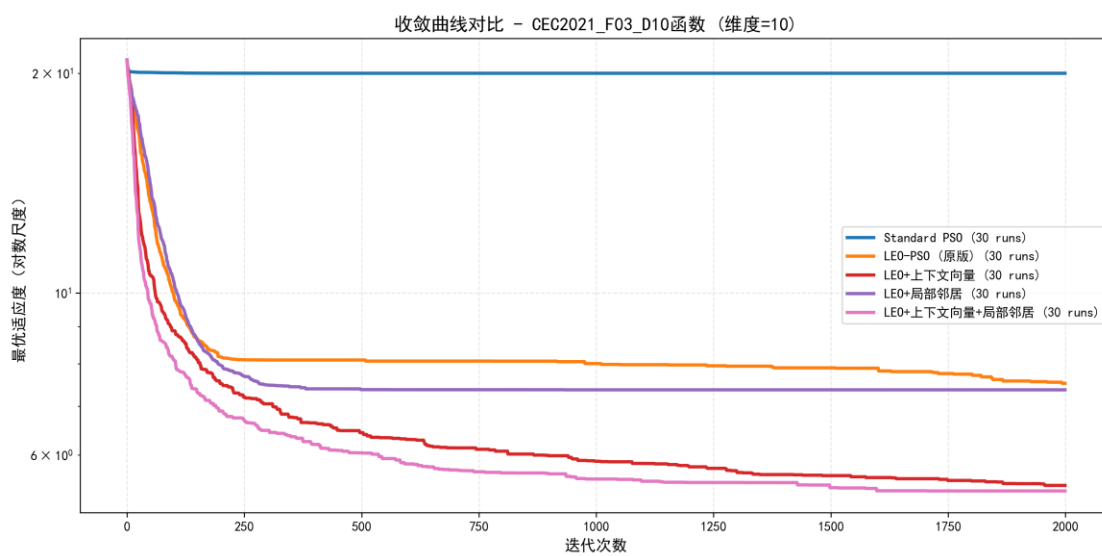
CEC2021_F02_D20 箱线图



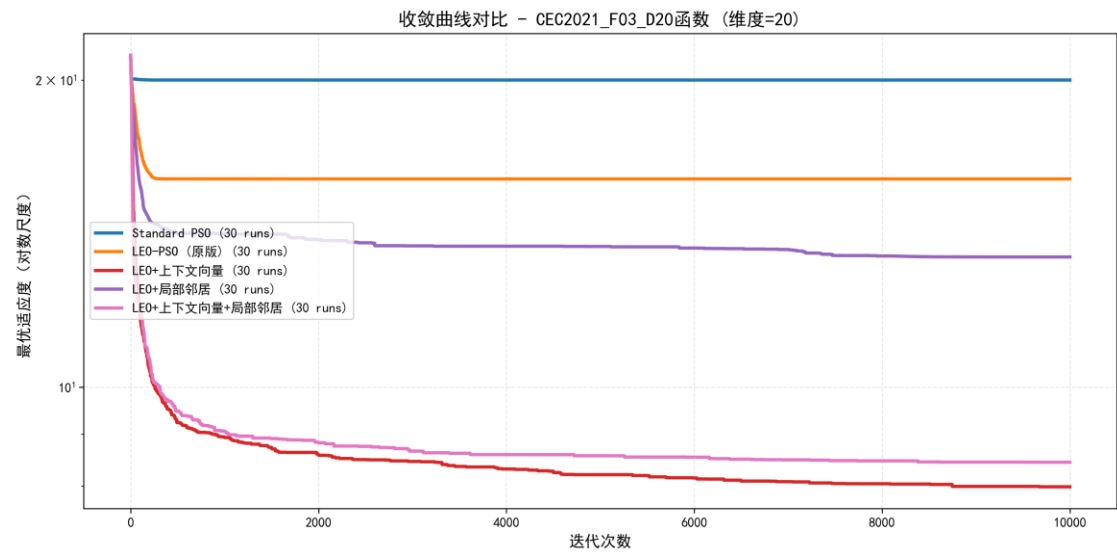
CEC2021_F02_D100 箱线图

5.1.3 CEC2021_F03

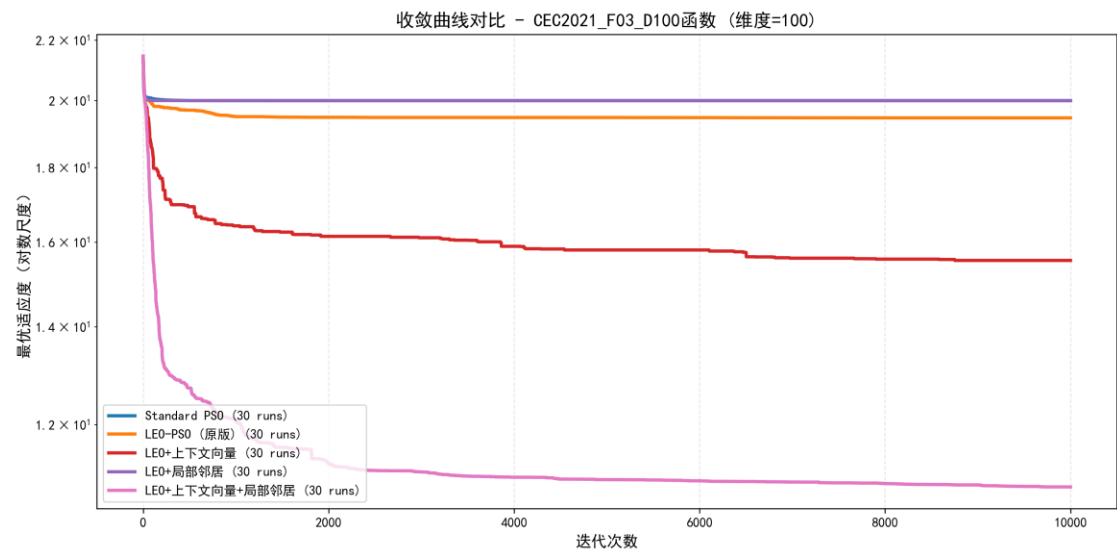
收敛曲线对比



CEC2021_F03_D10 收敛曲线

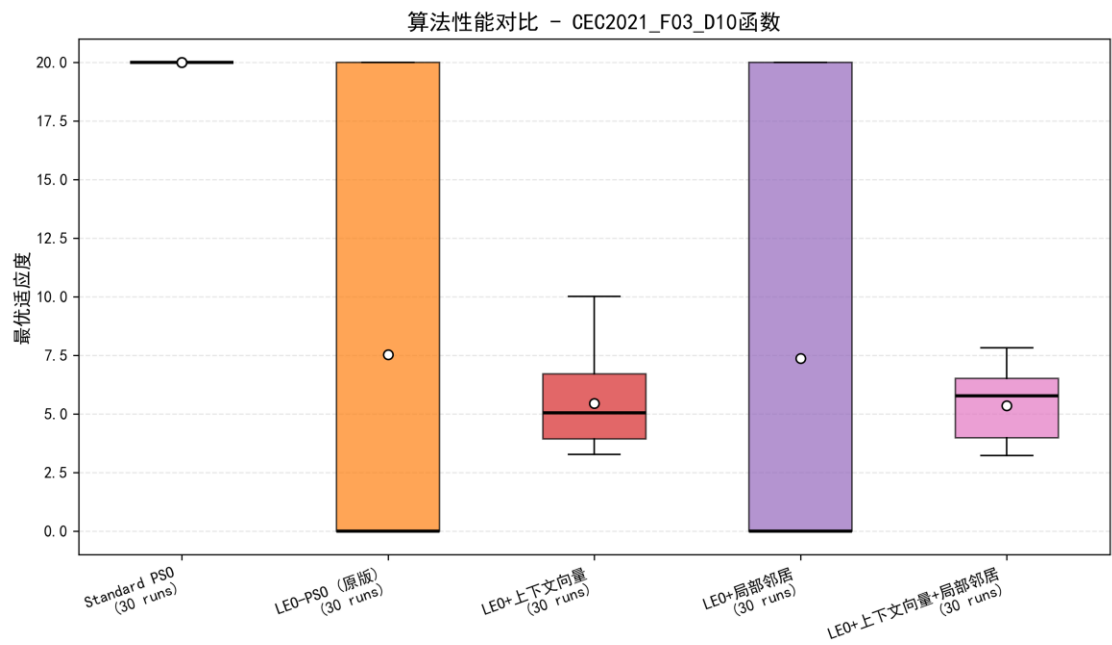


CEC2021_F03_D20 收敛曲线

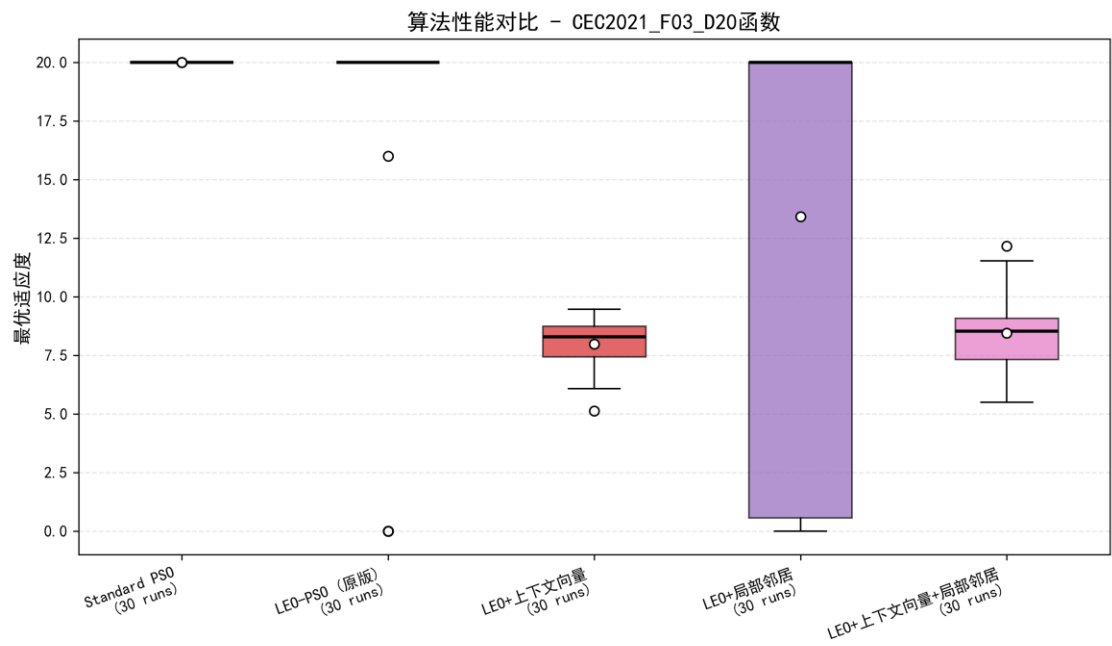


CEC2021_F03_D100 收敛曲线

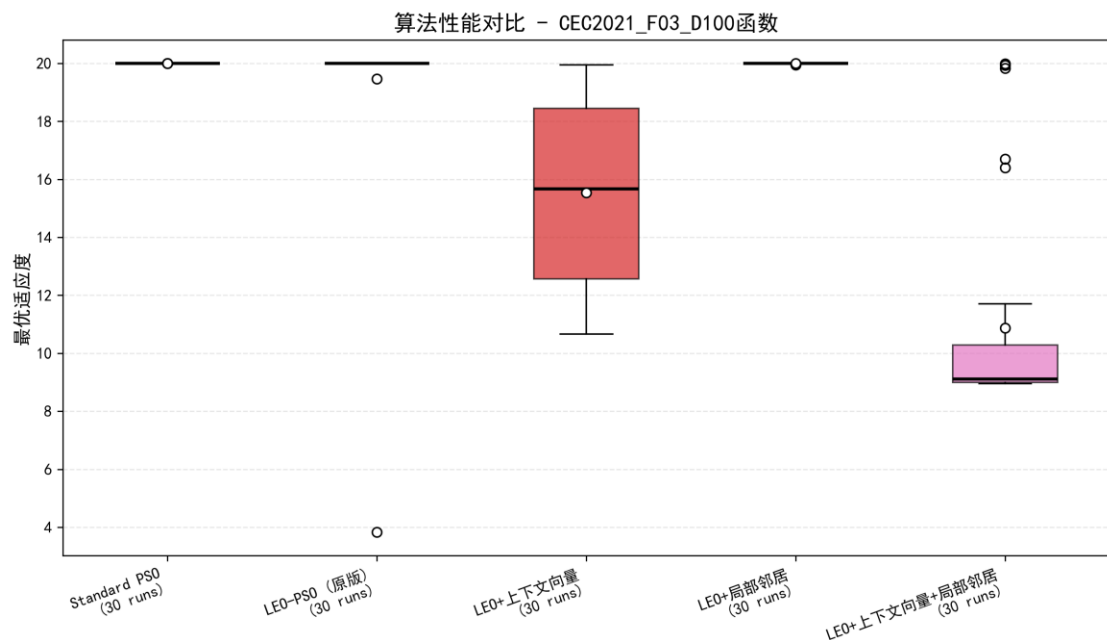
箱线图对比



CEC2021_F03_D10 箱线图



CEC2021_F03_D20 箱线图



CEC2021_F03_D100 箱线图

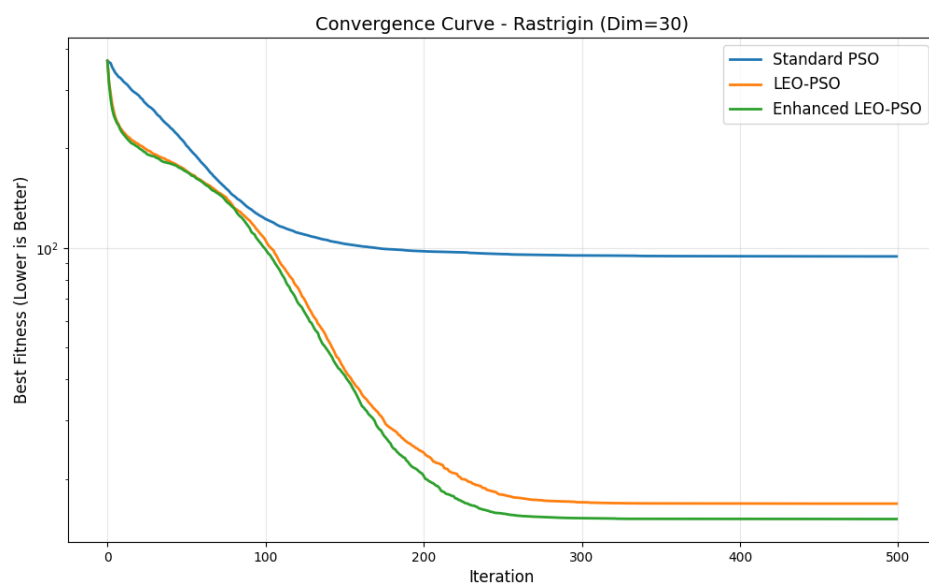
5.1.4 enhance_leo_pso (LEO_SRE_PSO) 实验结果

enhance_leo_pso 算法简介

enhance_leo_pso (技术名称为 LEO_SRE_PSO) 是在 LEO 基础上进行的探索和改进, 通过引入选择性反向经验机制, 同时存储正向和反向经验, 使 ANN 能够学习更全面的搜索策略。

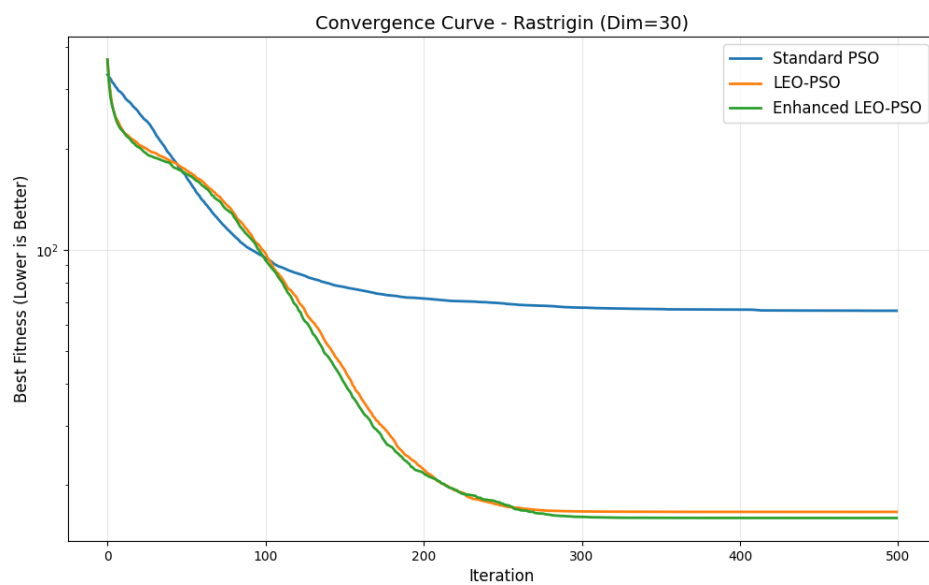
不同经验库大小的性能对比

本文测试了经验库大小为 300 和 500 的 enhance_leo_pso 算法, 实验结果分别如图 5.4.1 和图 5.4.2 所示。



enhance_leo_pso (经验库大小 300)

图 5.4.1 *enhance_leo_pso* (经验库大小 300)



enhance_leo_pso (经验库大小 500)

图 5.4.2 *enhance_leo_pso* (经验库大小 500)

结果分析

从图 5.4.1 和图 5.4.2 可以看出：

1. **性能提升：**enhance_leo_pso 算法在测试函数上的性能优于标准 PSO 和原版 LEO，验证了选择性反向经验机制的有效性。
2. **经验库大小的影响：**
 - 当经验库大小为 100、1000 时算法性能与 LEO 基本持平；
 - 当经验库大小为 300 时，enhance_leo_pso 能够显著提高算法的性能；
 - 当经验库大小增加到 500 时，算法的性能进一步提升，特别是在复杂的多模态函数上，优势更加明显。
 - 这表明适当增加经验库大小可以使 ANN 学习到更多的搜索经验，从而提高算法的性能。
3. **收敛速度：**enhance_leo_pso 的收敛速度明显快于标准 PSO 和原版 LEO，能够更快地接近最优解。
4. **稳定性：**enhance_leo_pso 的稳定性也优于其他算法，结果的分布更加集中，标准差更小。

与其他变种的结合

反向经验加入其他 LEO 变种后，性能略差于加入前框架，这表明选择性反向经验机制是一种有效的改进策略，但可能需要对反向经验进行一些筛选，过滤掉部分抖动和无效信息。

5.2 多目标实验结果

多目标实验结果显示，LEO 及其变种在 DTLZ1-DTLZ3 测试函数上的 IGD 值均小于标准 PSO，其中 LEO+上下文向量+局部邻居变种表现最佳。

5.3 结果分析

1. **收敛速度：**LEO 及其变种的收敛速度明显快于标准 PSO，尤其是在高维问题上，优势更加明显。这是因为 LEO 通过 ANN 学习正向经验，能够更快地找到最优解的方向。
2. **求解精度：**LEO 及其变种的求解精度明显高于标准 PSO，其中 LEO+上下文向量+局部邻居变种的求解精度最高。这是因为上下文向量提供了更丰富的信息，局部邻居信息增强了算法的局部搜索能力。
3. **稳定性：**LEO 及其变种的稳定性明显优于标准 PSO，箱线图显示其结果的分布更加集中，标准差更小。

4. **维度敏感性**: 随着维度的增加, 所有算法的性能都有所下降, 但 LEO 及其变种的性能下降幅度明显小于标准 PSO, 说明其在高维问题上具有更好的性能。
5. **组件影响**: 上下文向量和局部邻居信息都能提高 LEO 的性能, 两者结合的效果最好。这是因为上下文向量提供了全局信息, 局部邻居信息提供了局部信息, 两者互补, 共同提高了算法的性能。

6 结论与未来工作

6.1 结论

本文提出了基于学习辅助进化的粒子群算法 (LEO) 及其三种变种: LEO+上下文向量、LEO+局部邻居和 LEO+上下文向量+局部邻居。这些算法通过引入人工神经网络学习正向经验、上下文向量和局部邻居信息, 实现了对 PSO 的有效改进。

实验结果表明: 1. LEO 及其变种在收敛速度、求解精度和稳定性方面均优于传统 PSO; 2. 上下文向量和局部邻居信息都能提高 LEO 的性能, 两者结合的效果最好; 3. LEO 及其变种在高维问题上具有更好的性能, 对维度的敏感性较低。

6.2 未来工作

1. 探索更有效的经验学习机制, 如反向经验等;
2. 将 LEO 应用于实际工程问题, 如参数优化、路径规划等。

参考文献

Z. -H. Zhan, J. -Y. Li, S. Kwong and J. Zhang, "Learning-Aided Evolution for Optimization," in IEEE Transactions on Evolutionary Computation, vol. 27, no. 6, pp. 1794-1808, Dec. 2023, doi: 10.1109/TEVC.2022.3232776.

keywords: {Optimization;Artificial intelligence;Evolution (biology);Artificial neural networks;Learning systems;Problem-solving;Benchmark testing;Artificial neural network (ANN);differential evolution (DE);evolutionary computation (EC);learning-aided evolution;many-objective optimization;multiobjective optimization;particle swarm optimization (PSO);single-objective optimization},

J. Kennedy and R. Mendes, "Neighborhood topologies in fully informed and best-of-neighborhood particle swarms," in IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 36, no. 4, pp. 515-519, July 2006, doi: 10.1109/TSMCC.2006.875410.

keywords: {Particle swarm optimization;Social network services;Network topology;Emulation;Testing;Humans;Neural networks;Feedforward neural networks;Statistics;Informatics;Algorithms;optimization;particle swarm},

附录

附录 A 算法伪代码

A.1 LEO 伪代码

```
初始化粒子群位置 X 和速度 V
初始化个体最优解 pBest=X
计算每个粒子的适应度值 pBest_fit
初始化全局最优解 gBest 和 gBest_fit
初始化经验库 arch 为空
初始化人工神经网络 ANN
for 迭代次数 < 最大迭代次数:
    r = 随机数
    if r < 学习概率 lp:
        for 每个粒子 i:
            L_pBest = ANN(pBest[i])
            随机选择两个粒子 r1, r2
            newX = L_pBest +  $\alpha$ *(pBest[r1]-pBest[r2])
            交叉生成 final_X
            new_X = final_X
        else:
            更新粒子速度 V
            更新粒子位置 X
            new_X = X
        计算 new_X 的适应度值 new_fit
        更新个体最优解 pBest 和 pBest_fit
        更新全局最优解 gBest 和 gBest_fit
        将正向经验添加到经验库 arch
        使用 arch 训练 ANN
        记录收敛曲线
end for
```

A.2 LEO+上下文向量伪代码

```
初始化粒子群位置 X 和速度 V
初始化个体最优解 pBest=X
计算每个粒子的适应度值 pBest_fit
初始化全局最优解 gBest 和 gBest_fit
初始化经验库 arch 为空
初始化人工神经网络 ANN（输入维度为上下文向量维度）
for 迭代次数 < 最大迭代次数:
    构建每个粒子的上下文向量 contexts
    r = 随机数
    if r < 学习概率 lp:
        for 每个粒子 i:
            L_pBest = ANN(contexts[i])
            随机选择两个粒子 r1, r2
```

```

        newX = L_pBest +  $\alpha$ *(pBest[r1]-pBest[r2])
        交叉生成 final_X
        new_X = final_X
    else:
        更新粒子速度 V
        更新粒子位置 X
        new_X = X
    计算 new_X 的适应度值 new_fit
    更新个体最优解 pBest 和 pBest_fit
    更新全局最优解 gBest 和 gBest_fit
    将正向经验和上下文向量添加到经验库 arch
    使用 arch 训练 ANN
    记录收敛曲线
end for

```

A.3 LEO+局部邻居伪代码

```

初始化粒子群位置 X 和速度 V
初始化个体最优解 pBest=X
计算每个粒子的适应度值 pBest_fit
初始化全局最优解 gBest 和 gBest_fit
初始化经验库 arch 为空
初始化人工神经网络 ANN
构建局部邻居拓扑结构 neighborhood
for 迭代次数 < 最大迭代次数:
    r = 随机数
    if r < 学习概率 lp:
        for 每个粒子 i:
            L_pBest = ANN(pBest[i])
            从 neighborhood[i]中随机选择两个粒子 r1, r2
            newX = L_pBest +  $\alpha$ *(pBest[r1]-pBest[r2])
            交叉生成 final_X
            new_X = final_X
        else:
            更新粒子速度 V
            更新粒子位置 X
            new_X = X
    计算 new_X 的适应度值 new_fit
    更新个体最优解 pBest 和 pBest_fit
    更新全局最优解 gBest 和 gBest_fit
    将正向经验添加到经验库 arch
    使用 arch 训练 ANN
    记录收敛曲线
end for

```

A.4 LEO+ 上下文向量+局部邻居伪代码

```
初始化粒子群位置 X 和速度 V
初始化个体最优解 pBest=X
计算每个粒子的适应度值 pBest_fit
初始化全局最优解 gBest 和 gBest_fit
初始化经验库 arch 为空
初始化人工神经网络 ANN（输入维度为上下文向量维度）
构建局部邻居拓扑结构 neighborhood
for 迭代次数 < 最大迭代次数:
    构建每个粒子的上下文向量 contexts
    r = 随机数
    if r < 学习概率 lp:
        for 每个粒子 i:
            L_pBest = ANN(contexts[i])
            从 neighborhood[i]中随机选择两个粒子 r1, r2
            newX = L_pBest +  $\alpha$ *(pBest[r1]-pBest[r2])
            交叉生成 final_X
            new_X = final_X
        else:
            更新粒子速度 V
            更新粒子位置 X
            new_X = X
    计算 new_X 的适应度值 new_fit
    更新个体最优解 pBest 和 pBest_fit
    更新全局最优解 gBest 和 gBest_fit
    将正向经验和上下文向量添加到经验库 arch
    使用 arch 训练 ANN
    记录收敛曲线
end for
```

附录 B 实验数据

实验数据包括原始结果、统计数据、收敛曲线数据和分析报告，可在以下文件中找到：

- 原始结果：*_raw_results.npy
- 统计数据：*_stats.csv
- 收敛曲线数据：*_convergence_data.csv
- 分析报告：*_analysis_report.txt

附录 C 可视化代码

可视化代码见 test.py 文件中的 plot_convergence_curves、plot_boxplot 和 analyze_components 函数。

论文完成日期：2025 年 12 月 23 日 作者：2312167-刘振毫